

ТУ-Варна

**Документация
към
курсов проект
по ООП**

Изготвил: Емил Долчинков

Специалност: КС

Курс: 2; Група: 16

Фак. №: 24621842

Съдържание

Увод.....	3
Описание и идея на проекта.....	3
Цел и задачи на разработката	3
Използвани технологии.....	3
Функционални изисквания	4
Архитектура на приложението	5
Repository pattern.....	6
Singleton модел	6
Command Pattern	7
Реализация	9
Пакети	9
Логика	9
Заклучение	11

Увод

Описание и идея на проекта

Проектът представлява разработка на конзолна информационна система за управление на библиотека, базирана на обектно-ориентираното програмиране. Основната идея е да се осигури надеждно съхранение, обработка и управление на данни за наличните книги и регистрираните потребители.

Приложението функционира чрез интерфейс с команден ред (*Command Line Interface* - CLI) и работи с текстови файлове. При стартиране потребителят може да зарежда съдържанието на даден файл в паметта, да извършва операции над данните и при необходимост да записва промените обратно на диска. За сигурност системата разделя потребителите на клиенти и администратори с различни нива на достъп.

Цел и задачи на разработката

Основната цел на курсовия проект е практическото прилагане на принципите на обектно-ориентираното програмиране за изграждането на софтуерно устойчива информационна система.

За постигането ѝ са дефинирани следните задачи:

Проектиране на обектен модел: Създаване на класове за моделиране на книги (автор, заглавие, уникален номер, рейтинг и др.) и потребители (потребителско име, парола, права).

Работа с файлове: Реализиране на логика за коректно четене и запис на данни в определен файлов формат.

Контрол на достъпа: Ограничаване на административните команди (добавяне/премахване на книги и потребители) само за оторизирани акаунти.

Потребителски интерфейс: Създаване на интерактивен CLI режим, който обработва командите и параметрите и извежда ясни съобщения при грешка.

Използвани технологии

За реализацията на проекта са използвани следните инструменти:

Език за програмиране: *Java* – за цялостното изграждане на бизнес логиката и структурата от класове.

Система за контрол на версиите: *Git* (с отдалечено хранилище в *GitHub*) – за регулярни актуализации и проследяване на промените в кода по време на разработката.

Линк към гитхъб: <https://github.com/EmilDol/Library>

Функционални изисквания

Системата поддържа следния набор от команди:

- open – Отваря (или създава нов) .xml файл и изчита съдържанието му
- close – Затваря промените правени по данните без да ги записва във файл
- save – Запазва промените по данните в оригиналния файл
- save as – Запазва промените по данните в подаден файл
- help – Извежда списък с всички команди в системата
- exit – Затваря програмата
- login – Автентикира потребител
- logout – Излиза от профила на потребител
- books all – Изкарва всички книги
- books add – Добавя нова книга
- books find – Търси книга по дадено заглавие
- books sort – Сортира книгите по заглавие
- books remove – Изтрива дадена книга по подаден идентификационен номер
- users add – Добавя нов неадминистраторски акаунт в системата
- users remove – Изтрива акаунт от системата

Изисква се системата да работи с .xml файлове.

Архитектура на приложението

Разписаният сорс код в приложението следва няколко основни архитектурни решения, за да позволи лесна поддръжка и скалируемост на продукта. Най-основните архитектурни решения използван за качествено изготвяне на този и проект са следните:

Контейнер на зависимостите (Dependency injection container)

В рамките на проекта е използван Dependency Injection (DI) container — механизъм за управление на зависимостите между обектите в приложението. Това е един от основните принципи на съвременното обектно-ориентирано проектиране, който допринася значително за чистотата, разширяемостта и тестваемостта на кода.

При изграждането на по-сложно приложение различните класове неизбежно зависят един от друг. Без DI всеки клас сам създава инстанциите на обектите, от които се нуждае — което води до силна свързаност между компонентите. Това прави кода труден за поддръжка, тъй като промяна в един клас може да изисква промени на много други места. Освен това тестването на отделни компоненти се затруднява, защото те носят в себе си конкретни имплементации на зависимостите си.

DI контейнерът решава този проблем, като поема отговорността за създаването и управлението на обектите. Вместо даден клас сам да инстанциира зависимостите си, той просто ги декларира — и контейнерът се грижи да ги достави при нужда. Така класовете остават независими от конкретните имплементации и разчитат единствено на абстракции.

DI контейнерът функционира като централен регистър на всички обекти и зависимости в системата. При стартиране на приложението контейнерът се конфигурира — регистрират се всички класове и се описват връзките между тях. Когато даден компонент бъде поискан, контейнерът автоматично връща инстанция на поисканата абстракция.

Използването на DI контейнер в това приложение позволява лесна подмяна на слоя за работа с данните. В контейнерът се регистрират *репозиториата*, които се

използват за манипулация на данните от файловете. Благодарение на този подход можем много лесно и бързо да заменим изходния формат на файла или да адаптираме приложението към работа с база данни. Нужно е единствено да разпишем нов наследник на `IBookRepository` и `IUserRepository` и в `Setup()` метода да регистрираме тях вместо сегашните имплементации за работа с xml файл.

Други неща, за което е полезен контейнерът е да пази потребителската сесия, името на текущо заредения файл и други общи зависимости в приложението.

Repository pattern

Този архитектурен подход, както по-горе е споменато подпомага лесното подменяне на модула за работа с данните на приложението. Тази архитектура върви ръка за ръка с DI контейнерът (той позволява тази архитектура да е полезна). Създава се интерфейс `IRepository`, който бива наследен от всички останали класове и интерфейси. В стартирането на приложението тези репозитория се регистрират в DI контейнера и после при обръщане към данния слой на приложението го правим чрез абстракциите.

В случаят при нас съществуват и `IBookRepository`, чиято работа е да е базов интерфейс за всички репозитория работещи с книги, понеже runtime Java не разграничава generic типовете за различни (`IRepository<Book>` би било равностойно на `IRepository<User>`). Чрез съществуването на `IBookRepository` можем да си осигурим сигурност, че няма да се случи да се регистрират 2 имплементации на една абстракция, понеже използваме типът на интерфейса като ключ в Map.

Singleton модел

Singleton е един от най-разпространените design patterns в обектно-ориентираното програмиране. Целта му е да гарантира, че даден клас има точно една инстанция в рамките на цялото приложение, като същевременно осигурява глобална точка за достъп до нея.

В контекста на проекта singleton се използва за компоненти, които трябва да са споделени между различни части на системата — например DI контейнерът. Той е разработен така, че позволява само една единствена инстанция в цялото приложение да съществува. По този начин се гарантира, че навсякъде ще има актуалните регистрирани абстракции и т.н.

Command Pattern

Използваният подход в проекта следва утвърдения design pattern Command Pattern, който е част от групата на поведенческите шаблони за проектиране. Основната му идея е да капсулира всяка операция в самостоятелен обект, като по този начин отделя извикващия код от конкретната логика на изпълнение.

Архитектурата се гради върху няколко основни елемента. На първо място стои абстракцията — интерфейсът `ICommand`, който дефинира общия договор за всички команди в системата. Всяка конкретна операция е реализирана като отделен клас, който имплементира този интерфейс. Така системата работи единствено с абстракцията, без да се интересува коя конкретна команда се изпълнява в момента.

На второ място стои `CommandFactory` — компонентът, отговорен за създаването на командите. Той приема `enum` стойност, която еднозначно идентифицира желаната операция, и връща съответната инстанция. Цялото знание за това кой клас стои зад кой `enum` е централизирано в една точка — самата фабрика. Това е реализация на допълнителния pattern `Factory`, който в случая работи в синергия с `Command Pattern`.

Най-голямото предимство на тази архитектура е, че основният управляващ цикъл на приложението остава изключително прост и непроменящ се. Той чете въведена команда, получава инстанция от фабриката, проверява изискванията чрез `RequiresLogin()`, `RequiresAdmin()` и `RequiresLogout()`, и при успех извиква `Execute()`. Добавянето на нова команда в системата не налага промяна на този цикъл — достатъчно е да се създаде нов клас и да се регистрира във фабриката.

Освен това всяка команда носи в себе си собствените си изисквания за достъп, което елиминира нуждата от централна `if-else` логика за проверка на права. Системата е затворена за модификация, но отворена за разширение — което е същността на `Open/Closed` принципа от `SOLID`.

Тук обаче имаме един проблем. Понеже искаме командите да не се занимават с това да обработват потребителски вход искаме да получават данните си по някакъв друг начин. Решението на този проблем е `CommandContext` класа. Той се приема като параметър от `Execute` метода на командите. Реално този клас е обвивка около речник с ключ низ и стойност `Object`. По този начин можем да подаваме всякакви данни като

контекст на командата като регистрираме нов запис в речника с даден ключ и подадем стойността, която ни е нужна.

Реализация

Пакети

Проектът е разработен в няколко основни пакета. Тяхното предназначение е да разделят класовете и интерфейсите в отделни части, за да е по-организирано и по-четимо. По този начин е много по-лесно външен човек да се ориентира из сорс кода и да разширява или редактира логика. Пакетите в приложението са:

- **Commands** – В този пакет са имплементирани всички командни класове и класовете отговарящи за работата с командите (ICommand, Factory класа и enum-a)
- **Container** – В този пакет е реализиран singleton DI контейнер
- **Context** – В този пакет е имплементиран класа за потребителската сесия в приложението
- **Data.Models** – Тук са разписани моделите за данните (Book и User)
- **Data.Repositories** – Тук са разписани конкретните имплементации на абстракциите върху Repository pattern-a
- **Data.Repositories.Contracts** – тук са разписани всички абстракции върху Repository pattern-a

Логика

За да стане по-ясно ще проследим логиката зад командата за автентикация (Login). Логиката зад командата следва следните стъпки:

1. Приемане на вход и определяне на командата: Приложението очаква вход от потребителя (командата), който в случая приемаме, че ще е login.
2. Създаваме променливите от тип ICommand и CommandContext, които ще инициализираме и попълним по-късно.
3. В main метода на програмата е разписан switch case оператор. Там са изредени всички възможни команди, които са поддържани от проекта. В case “login” виждаме, че изискваме от factory класа да ни върне инстанция на ICommand по даден код CommandCode.Login. Това ни връща инстанция на LoginCommand класа от пакета commands.

4. Подканваме потребителя да въведе потребителско име и парола, с които да влезе в профила, като ги добавяме в `CommandContext` класа за имената „username“ и „password“.

5. След това излизаме от `switch case` оператора и продължаваме напред. Минаваме проверки за това дали имаме зареден файл. Правим го посредством контейнерът. В него пазим променлива за това дали има зареден файл. Ако тя е `false` и командата, която сме създали сега не е от тип `OpenCommand` или `HelpCommand` значи се опитваме да извършим нещо забранено.

6. След това проверяваме дали имаме вече зареден файл и се опитваме да заредим нов. Това не е позволено. Нужно е да затворим сегашния файл.

7. Минават проверки по изискванията за `login`, `logout` и админ права, които са запазени в `command` класа

8. Стига ред на самата логика на командата. Извикваме `Execute` метода на `ICommand` променливата (в случая инстанция на `LogInCommand`).

9. Тук изкарваме потребителското име и паролата от `context` параметъра.

10. Взимаме `IUserRepository` инстанция от контейнера.

11. Търсим потребител с даденото потребителско име в данните ни.

12. Ако не намерим връщаме `false` и прекратяваме изпълнението на командата.

13. Ако намерим проверяваме дали паролата на потребителя от репозиторието съвпада с паролата подадена през контекста.

14. Ако не – връщаме провал и прекратяваме командата.

15. Ако да – успех! Взимаме сесията от контейнера и запазваме потребителя вътре.

Всички останали команди следват подобни стъпки на изпълнение като се заместват стъпките изпълняващи конкретна бизнес логика.

Заклучение

Разработката покрива нуждите на една система за поддръжка на библиотека. Позволява менажирането на книги и потребители. Същевременно е разписана по добър, разбираем, „развързан“ и скалируем начин. Всеки модул лесно може да се подмени с друг без да се разбутва много сорс код.

Разбира се има още много възможности за подобрение както при всеки проект. Последващи стъпки за разработка биха включвали:

- Внедряването на factory класа за командите в DI контейнерът.
- Разработката на база данни, която да премахне нуждата от работата с xml файлове, които са по-податливи на промяна и загуба на данни.
- Инкорпорирането на база данни би се случила с минимални усилия и промени по сорс кода, както по-горе е споменато. В момента всичко, което работи с данни използва и работи само с абстракциите IUserRepository И IBookRepository. Това от своя страна позволява мигрирането към друг вид съхранение на данните в 2 прости стъпки: имплементиране на наследници на двете абстракции и регистрирането им в DI контейнерът вместо сегашните.
- Разписване на по-обстойни проверки и валидации на потребителския вход.